Київський національний університет імені Тараса Шевченка

Кафедра мережевих і Інтернет технологій

Лабораторна робота №7-8

з дисципліни

«Технології створення програмних продуктів»

ШАБЛОН COMPOSITE.

ШАБЛОН DECORATOR І FACADE.

ШАБЛОН FLYWEIGHT (ПРИСТОСУВАНЕЦЬ)

Група: МІТ-м11

Студент: Переверзев Олексій

Київ 2026

Завдання 1: Шаблон Composite

Створіть ієрархію класів для графічного редактора, де Shape є базовим інтерфейсом, а CompositeShape може містити кілька об’єктів Shape.

Код реалізації:

```
using System;
using System.Collections.Generic;

namespace CompositeTask
{
    public interface IShape // Component
    {
        void Add(IShape shape);
        void Remove(IShape shape);
        void Draw(int depth);
    }

    public class Circle : IShape // Leaf
    {
        public void Add(IShape shape)
        {
            Console.WriteLine("Неможливо додати до простого Кола");
        }

        public void Remove(IShape shape)
        {
            Console.WriteLine("Неможливо видалити з простого Кола");
        }

        public void Draw(int depth)
        {
            Console.WriteLine(new String('-', depth) + " Коло");
        }
    }

    public class Square : IShape
    {
        public void Add(IShape shape)
        {
            Console.WriteLine("Неможливо додати до простого Квадрата");
        }

        public void Remove(IShape shape)
        {
            Console.WriteLine("Неможливо видалити з простого Квадрата");
        }

        public void Draw(int depth)
        {
            Console.WriteLine(new String('-', depth) + " Квадрат");
        }
    }

    public class CompositeShape : IShape // Composite
    {
        private List<IShape> _shapes = new List<IShape>();

        public void Add(IShape shape)
        {
            _shapes.Add(shape);
        }
```

```
        public void Remove(IShape shape)
        {
            _shapes.Remove(shape);
        }

        public void Draw(int depth)
        {
            Console.WriteLine(new String('-', depth) + " Складена фігура:");
            foreach (var shape in _shapes)
            {
                shape.Draw(depth + 2);
            }
        }
    }

    class Program
    {
        static void Main()
        {
            Console.WriteLine("=== 1. Шаблон Composite (Прозорий підхід) ===\n");

            IShape rootShape = new CompositeShape();
            IShape circle = new Circle();

            rootShape.Add(circle);

            IShape group = new CompositeShape();
            group.Add(new Square());
            group.Add(new Circle());

            rootShape.Add(group);

            rootShape.Draw(1);

            Console.WriteLine("\n--- Перевірка реакції простої фігури (Leaf) ---");
            circle.Add(new Square());

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== 1. Шаблон Composite (Прозорий п?дх?д) ===

- Складена ф?гура:
--- Коло
--- Складена ф?гура:
----- Квадрат
----- Коло

--- Перев?рка реакц?ї простої ф?гури (Leaf) ---
Неможливо додати до простого Кола
```

Реалізовано шаблон Composite для об'єднання об'єктів у деревоподібну структуру. Складені фігури (CompositeShape) та прості фігури (Circle, Square) реалізують спільний інтерфейс IShape. Це дозволяє клієнтському коду звертатися як до окремих фігур, так і до їхніх груп абсолютно однаково, викликаючи метод Draw().

Завдання 2: Шаблон Decorator

Реалізуйте клас IceCreamDecorator, який додасть нові інгредієнти до базового класу IceCream без зміни його коду.

Код реалізації:

```
using System;

namespace DecoratorTask
{
    public abstract class IceCream // Component
    {
        public abstract string GetDescription();
        public abstract double GetCost();
    }

    public class BaseIceCream : IceCream // ConcreteComponent
    {
        public override string GetDescription() => "Вершкове морозиво";
        public override double GetCost() => 50.0;
    }

    public abstract class IceCreamDecorator : IceCream // Decorator
    {
        protected IceCream _iceCream;

        public IceCreamDecorator(IceCream iceCream)
        {
            _iceCream = iceCream;
        }

        public override string GetDescription() => _iceCream.GetDescription();
        public override double GetCost() => _iceCream.GetCost();
    }

    public class ChocolateDecorator : IceCreamDecorator // ConcreteDecorator
    {
        public ChocolateDecorator(IceCream iceCream) : base(iceCream) { }

        public override string GetDescription() => _iceCream.GetDescription() + ", з
шоколадом";
        public override double GetCost() => _iceCream.GetCost() + 15.0;
    }

    public class CaramelDecorator : IceCreamDecorator
    {
        public CaramelDecorator(IceCream iceCream) : base(iceCream) { }

        public override string GetDescription() => _iceCream.GetDescription() + ", з
карамеллю";
        public override double GetCost() => _iceCream.GetCost() + 10.0;
    }

    class Program
    {
        static void Main()
        {
            Console.WriteLine("=== Шаблон Decorator ===");
            IceCream myIceCream = new BaseIceCream();
            myIceCream = new ChocolateDecorator(myIceCream);
            myIceCream = new CaramelDecorator(myIceCream);
```

```
            Console.WriteLine($"Склад: {myIceCream.GetDescription()}");
            Console.WriteLine($"Вартість: {myIceCream.GetCost()} грн\n");
            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== Шаблон Decorator ===
Склад: Вершкове морозиво, з шоколадом, з карамеллю
Варт?сть: 75 грн
```

Використано шаблон Decorator для динамічного розширення функціональності об'єкта IceCream. Замість створення множини підкласів (наприклад, "МорозивоЗШоколадомТаКарамеллю"), додавання інгредієнтів відбувається шляхом послідовного обгортання базового об'єкта конкретними декораторами. Це забезпечує максимальну гнучкість системи.

Завдання 3: Шаблон Facade

Створіть клас ComputerFacade, який приховає складність запуску комп’ютера за допомогою простого методу turnOn().

Код реалізації:

```
using System;

namespace FacadeTask
{
    public class CPU
    {
        public void Freeze() => Console.WriteLine("CPU: Заморожування процесів...");
        public void Jump(long position) => Console.WriteLine($"CPU: Перехід за
адресою {position}...");
        public void Execute() => Console.WriteLine("CPU: Виконання інструкцій...");
    }

    public class Memory
    {
        public void Load(long position, string data) => Console.WriteLine($"Memory:
Завантаження даних '{data}' в позицію {position}...");
    }

    public class HardDrive
    {
        public string Read(long lba, int size) => "ОС Windows";
    }

    public class ComputerFacade
    {
        private CPU _cpu;
        private Memory _memory;
        private HardDrive _hardDrive;

        public ComputerFacade()
        {
            _cpu = new CPU();
```

```
            _memory = new Memory();
            _hardDrive = new HardDrive();
        }

        public void turnOn()
        {
            Console.WriteLine("--- Початок запуску комп'ютера ---");
            _cpu.Freeze();
            string bootSector = _hardDrive.Read(0, 1024);
            _memory.Load(0, bootSector);
            _cpu.Jump(0);
            _cpu.Execute();
            Console.WriteLine("--- Комп'ютер успішно запущено ---");
        }
    }

    class Program
    {
        static void Main()
        {
            Console.WriteLine("=== Шаблон Facade ===");
            ComputerFacade computer = new ComputerFacade();
            computer.turnOn();

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== Шаблон Facade ===
--- Початок запуску комп'ютера ---
CPU: Заморожування процес?в...
Memory: Завантаження даних 'ОС Windows' в позиц?ю 0...
CPU: Перех?д за адресою 0...
CPU: Виконання ?нструкц?й...
--- Комп'ютер усп?шно запущено ---
```

Створено фасадний клас ComputerFacade, який надає єдину точку входу для роботи зі складною підсистемою комп'ютера (процесор, пам'ять, жорсткий диск) . Це дозволяє клієнтському коду (метод Main) виконати складний ланцюжок дій викликом лише одного простого методу turnOn(), повністю приховуючи від нього внутрішню реалізацію .

Завдання 4: Шаблон Flyweight

Реалізуйте систему для книгарні, де клас Book використовується для зберігання інформації про книги, а клас BookFlyweight уникне дублювання даних для книг з однаковими авторами та назвами.

Код реалізації:

```
using System;
using System.Collections.Generic;

namespace FlyweightTask
{
```

```
    public class BookFlyweight // Flyweight
    {
        public string Title { get; private set; }
        public string Author { get; private set; }

        public BookFlyweight(string title, string author)
        {
            Title = title;
            Author = author;
            Console.WriteLine($"[Фабрика] Створено новий еталон книги: '{Title}'
({Author})");
        }
    }

    public class BookFlyweightFactory // Flyweight Factory
    {
        private Dictionary<string, BookFlyweight> _flyweights = new
Dictionary<string, BookFlyweight>();

        public BookFlyweight GetBookFlyweight(string title, string author)
        {
            string key = title + "_" + author;
            if (!_flyweights.ContainsKey(key))
            {
                _flyweights[key] = new BookFlyweight(title, author);
            }
            return _flyweights[key];
        }
    }

    public class Book
    {
        private BookFlyweight _flyweight;
        public string InventoryNumber { get; set; }

        public Book(BookFlyweight flyweight, string inventoryNumber)
        {
            _flyweight = flyweight;
            InventoryNumber = inventoryNumber;
        }

        public void DisplayInfo()
        {
            Console.WriteLine($"Інвентарний №: {InventoryNumber} | Книга:
'{_flyweight.Title}', Автор: {_flyweight.Author}");
        }
    }

    class Program
    {
        static void Main()
        {
            Console.WriteLine("=== Шаблон Flyweight ===");
            BookFlyweightFactory factory = new BookFlyweightFactory();
            List<Book> library = new List<Book>();

            BookFlyweight hpFlyweight = factory.GetBookFlyweight("Гаррі Поттер",
"Дж. Роулінг");

            library.Add(new Book(hpFlyweight, "INV-001"));
            library.Add(new Book(hpFlyweight, "INV-002"));
            library.Add(new Book(hpFlyweight, "INV-003"));

            BookFlyweight lotrFlyweight = factory.GetBookFlyweight("Володар
Перснів", "Дж.Р.Р. Толкін");
```

```
            library.Add(new Book(lotrFlyweight, "INV-004"));

            Console.WriteLine("\nСписок книг у книгарні:");
            foreach (var book in library)
            {
                book.DisplayInfo();
            }

            Console.ReadKey();
        }
    }
}
```

Результат виконання програми:

```
=== Шаблон Flyweight ===
[Фабрика] Створено новий еталон книги: 'Гарр? Поттер' (Дж. Роул?нг)
[Фабрика] Створено новий еталон книги: 'Володар Персн?в' (Дж.Р.Р. Толк?н)

Список книг у книгарн?:
?нвентарний №: INV-001 | Книга: 'Гарр? Поттер', Автор: Дж. Роул?нг
?нвентарний №: INV-002 | Книга: 'Гарр? Поттер', Автор: Дж. Роул?нг
?нвентарний №: INV-003 | Книга: 'Гарр? Поттер', Автор: Дж. Роул?нг
?нвентарний №: INV-004 | Книга: 'Володар Персн?в', Автор: Дж.Р.Р. Толк?н
```

Застосовано шаблон Flyweight для оптимізації використання оперативної пам'яті. Незмінні дані (внутрішній стан: назва та автор книги) винесено в об'єкт-пристосуванець BookFlyweight . Фабрика слідкує за тим, щоб для однакових книг такий об'єкт створювався лише один раз і спільно використовувався всіма унікальними екземплярами Book .

Висновки: під час роботи досліджено структурні шаблони проектування, які допомагають оптимізувати архітектуру та взаємодію об'єктів у складних системах. Composite дозволяє працювати з деревоподібними структурами та окремими об'єктами через єдиний універсальний інтерфейс. Decorator забезпечує гнучке динамічне додавання нових властивостей до об'єктів без необхідності створення розгалуженої ієрархії підкласів. Facade надає простий і зрозумілий інтерфейс для доступу до складної внутрішньої підсистеми, мінімізуючи залежності. Flyweight суттєво зменшує споживання оперативної пам'яті за рахунок спільного використання незмінного стану у великій кількості дрібних об'єктів.